

Comparative Study of KV Cache Optimization Techniques for Large Language Models

Kiyan Amirian **Nicholas Forman** **Helia Hosseini** **Hengyuan Qi**
kamirian@umd.edu nforman@umd.edu helia@umd.edu hqi@umd.edu

1 Problem statement

One of the major challenges in modern large language models (LLMs) based on the Transformer architecture is the high memory cost of inference. During autoregressive decoding, each Transformer layer produces key and value tensors for the attention mechanism, which are cached to avoid recomputation at every decoding step. This key-value (KV) cache substantially improves computational efficiency by enabling the reuse of previously computed representations. However, as LLMs scale in both model size and context length, the KV cache itself becomes a major bottleneck. Each input sequence in a batch requires its own cache, and every newly generated token further enlarges the stored key and value tensors. As a result, the memory footprint of the KV cache can rival or even exceed the size of the model parameters. For example, a model with a batch size of 64 and a context length of 4096 may require a KV cache that occupies more than three times the memory needed to store the model weights (He et al., 2024). This excessive memory consumption limits achievable batch sizes, restricts usable context lengths, and increases inference costs, particularly on memory-constrained GPU hardware.

To address this challenge, recent research has proposed a range of KV cache optimization techniques aimed at reducing memory usage and computational overhead while preserving model accuracy. These approaches include token selection and eviction methods, which selectively retain only the most informative cached tokens, as well as quantization and compression techniques that reduce the precision or representation size of cached tensors. The goal of this project is to study and systematically compare modern KV cache optimization methods, evaluating their ability to reduce cache size and in-

ference overhead without significantly degrading accuracy. Specifically, we examine token selection and eviction approaches (Sliding Window and StreamingLLM (Xiao et al., 2023)), quantization and compression methods (KIVI (Liu et al., 2024) and ZipCache (He et al., 2024)), and compare them against baseline strategies, including the full KV cache and sliding-window caching. All methods are applied to the LLaMA-2-7B model (Touvron et al., 2023) and evaluated across multiple benchmark datasets, CNN/DailyMail (See et al., 2017), GSM8K (Grade School Math 8K dataset for LLM) (Cobbe et al., 2021), and CoQA (Conversational Question Answering) (Reddy et al., 2019), to assess their performance across summarization, reasoning, and question-answering tasks.

2 What we proposed vs. what we accomplished

Below we summarize the main objectives proposed in our project proposal and indicate their completion status. Any deviations from the original plan are briefly explained.

- **Study KV cache memory bottlenecks in Transformer-based LLM inference**

Status: Completed. We analyzed KV cache growth with respect to batch size and context length and profiled its impact on GPU memory usage during inference.

- **Reproduce and evaluate token-level KV cache optimization methods (H2O, KIVI, KVZip)**

Status: Partially completed. We successfully implemented and evaluated KIVI, ZipCache-style quantization, and StreamingLLM-style token eviction. Full KVZip-style merging was not implemented due to the complexity

of its reconstruction mechanism. H2O was not finished by a group member not associated with this project anymore.

- **Evaluate models across multiple architectures (GPT-2, LLaMA-2-7B, and LLaMA-2-Chat)**

Status: Partially completed. We initially evaluated both the pretrained and chat-tuned variants of LLaMA-2-7B; however, the pretrained model did not perform reliably on long, instruction-following prompts. Consequently, we focused our experiments on the supervised fine-tuned (SFT) chat model, LLaMA-2-7B-Chat, which better aligns with the conversational and summarization tasks considered. Evaluation on additional model architectures was further constrained by limited computational resources.

- **Benchmark methods on long-context language modeling datasets (WikiText, PG19)**

Status: Partially completed. While we did not evaluate on WikiText or PG19, we instead benchmarked the methods on CNN/DailyMail, GSM8K, and CoQA. This shift was driven both by a change in model selection, moving to SFT chat model that performs more reliably on complex, instruction-following tasks, and by the desire to evaluate task-level accuracy and reasoning degradation under KV cache compression. Notably, new datasets involve multi-turn conversational context and long context accumulation, partially fulfilling the original objective of long-context evaluation.

- **Compare against strong baselines (full KV cache, sliding-window cache, uniform quantization)**

Status: Partially completed. We evaluated against strong baselines including the full KV cache and sliding-window cache across all experiments. Uniform quantization was not implemented due to time and compute constraints.

- **Perform cross-method and hybrid analysis (e.g., pruning combined with quantization)**

Status: Not completed. Due to time and compute constraints, optimization methods were

evaluated independently rather than in combination.

3 Related work

The attention mechanism (Vaswani et al., 2017) is a fundamental structure in most of the current large language models. Through the introduction of the q, k, v triple, the language model learns to pay attention to the correct location of the crucial information needed to predict the new token. A great benefit of this attention structure is that in the training process, the computations of the hidden state h_i 's at different positions are independent of each other, and thus could be completely parallelized to achieve a higher efficiency. However, during inference time when the model is trying to generate new tokens, the process becomes sequential in the sense that the token x_i cannot be generated without first knowing x_{i-1} . This sequential decoding would not be a big issue if the length of the response is short. However, in real world applications, the prompts and responses could be very long, meaning that computing all the inner products and sums for the q, k, v vectors at each step is nearly infeasible. Luckily, by saving the old KV-pairs into memory, we could reuse them at each new generation step. This method is referred as the full KV cache, and it is the default method incorporated by a lot of real world LLMs.

However, with the use of the KV cache, a new problem is introduced. Modern language models consist of many layers of multi-head self-attention blocks, causing the KV cache to occupy a substantial portion of memory during inference, with its size growing linearly with the generated sequence length. Moreover, in latency-sensitive settings, even with KV caching enabled, the memory bandwidth and compute overhead of attending over long contexts can significantly limit inference throughput. To address these challenges, a growing body of work on KV cache optimization has emerged (Li et al., 2024). Under this paradigm, the goal is to simplify or compress the KV cache to reduce memory footprint and accelerate decoding while incurring minimal accuracy degradation. For example, StreamingLLM (Xiao et al., 2023) dynamically shrinks the KV cache by retaining only a sliding window of recent tokens and a small set of global tokens. Building on this idea, H2O (Zhang et al., 2023) uses heuristic policies to retain tokens with high attention head outputs. From a quantization perspective, ZipCache (He

et al., 2024) applies mixed-precision compression by preserving salient KV entries in higher precision while aggressively quantizing less important tokens. KIVI (Liu et al., 2024) further advances this direction through tuning-free asymmetric 2-bit quantization with per-channel and per-token scaling, enabling drop-in deployment without re-training. More recently, KVZip (Kim et al., 2025) proposes a query-agnostic structural compression scheme that merges redundant KV entries and reconstructs context on demand.

More recent approaches explore structured sparsity and grouping-based compression to further reduce KV redundancy. Mustafar (Joo et al., 2025) promotes unstructured sparsity in the KV cache by pruning low-magnitude entries at a fine granularity, enabling direct attention computation over compressed representations with minimal quality loss. ChunkKV (Liu et al., 2025) exploits temporal locality by grouping tokens into chunks and sharing KV representations within each chunk, reducing both memory footprint and attention cost for long-context decoding. Other recent work adopts structure-aware KV compression, such as Eigen Attention (Saxena et al., 2024), which uses low-rank projections of KV states, applying cross-layer SVD to eliminate redundant KV representations, highlighting a trend toward hardware-friendly KV cache designs. The next section discusses several of these approaches in more detail, as they are implemented and evaluated in our project for comparative analysis.

4 Baselines

To evaluate the effectiveness of KV cache optimization techniques, we compare all proposed methods against two common and easy-to-implement baselines: the *full KV cache* and the *sliding-window KV cache*. The full KV cache serves as a natural reference point, as it preserves the entire input and generation history in memory and therefore provides the model with maximal context at every decoding step. While this approach yields the strongest performance for a fixed model and prompt, it also incurs the highest memory cost, with cache size growing linearly in both input length and the number of generated tokens. This rapid growth in memory usage is precisely what motivates the development of KV cache compression and eviction methods, making the full KV cache a critical upper-bound baseline

for comparison.

The sliding-window KV cache provides a contrasting baseline that prioritizes memory efficiency over full-context preservation. Instead of retaining all past tokens, this approach maintains a fixed-size window of the most recent tokens in the cache and discards older entries as generation proceeds. By bounding the cache size, the sliding-window strategy significantly reduces memory usage and enables longer sequences to be processed on limited hardware. However, this comes at the cost of losing long-range contextual information, which can be important for tasks such as document summarization, reasoning, and instruction following. As a result, sliding-window caching often leads to degraded output quality relative to the full KV cache, particularly on tasks that rely on global context. Nonetheless, the sliding-window KV cache serves as a strong baseline for token-eviction methods such as StreamingLLM (Xiao et al., 2023), as it implements a simple, deterministic eviction policy based solely on token recency.

The full KV cache baseline does not introduce any cache-specific hyperparameters, as it retains all key and value tensors generated during inference. The sliding-window baseline introduces a single hyperparameter: the window size, W , which determines the number of most recent tokens retained in the cache. In all experiments, we fix the window size to $W = 256$. This window size was chosen through experimentation using the CNN/DailyMail dataset. We initially tried a window size of 512, but realized that the window size may have been too large for the CNN dataset since it has an average token length of 781. Thus, for some examples, the sliding window would mimic the full KV cache as the window size was large enough to take in the entire article. We then shrunk the window size to 128, but observed that this window size may not be large enough to gather enough context for an accurate summary. Thus, we chose $W = 256$, as it is not large enough to mimic the full KV cache, but not small enough to produce poor quality generations. We then kept this window size consistent across the datasets to keep consistency.

Our study is conducted entirely in an inference-only setting, and no models are trained or fine-tuned as part of this work. As a result, there is no train or validation split in the traditional sense. All evaluations are performed on the official test

splits of CNN/DailyMail, GSM8K, and CoQA as provided by the HuggingFace `datasets` library. No hyperparameters were tuned on the test set, and all decoding and cache configurations were fixed prior to evaluation and held constant across methods.

5 Methods

Our project studies several representative methods for KV cache compression in long-context LLM inference, focusing on how different design choices affect memory footprint and downstream task accuracy. We explore four state-of-the-art approaches, **StreamingLLM**, **KIVI**, **ZipCache**, and **H2O**, each of which reflects a distinct philosophy for managing KV cache growth during inference.

To keep the implementation effort modular and reproducible, each method is implemented and analyzed independently by a different team member. This section therefore, consists of method-specific subsections, where each subsection describes the design rationale, implementation details, and limitations of a single approach. This structure allows us to compare the methods under a shared evaluation framework while maintaining clear ownership of individual contributions.

5.1 StreamingLLM

The StreamingLLM model (Xiao et al., 2023) attempts the KV cache optimization problem by modifying the elements of the KV cache. The idea is that the most recent tokens are more helpful for predicting the next token than the more distant ones. Instead of the full cache, we could only save the KV pairs of the W most recent tokens. In this way, the size of the KV cache is reduced from being linear to the response length to a constant W . Moreover, the paper also observes that the first several tokens of a response are incredibly helpful for keeping the accuracy of text generation. Thus, Streaming LLM proposes that we keep the KV pairs for the first s and last W tokens in the cache while discarding all other KV pairs in between. The authors argue that with this set up, the performance of the model is largely kept, and in particular it’s significantly better than only using the last W KV pairs in the form of a sliding window. In the paper, s is usually taken to be 4 and is also called “attention sink”. Setting to be 512 in default, W is a more versatile parameter, and could be decided based on the specific

dataset. In our implementation, we stay consistent with the window size we chose for sliding window and keep $W = 256$, and set $s = 8$ to compensate for the smaller window size.

The ideas and implementation underlying StreamingLLM are relatively straightforward, making it a natural starting point for implementation and comparison against baseline methods. However, as noted earlier, one of the central advantages of the attention mechanism is its ability to attend to arbitrarily distant tokens. By discarding most intermediate KV pairs, StreamingLLM implicitly assumes that retaining the first s tokens is sufficient to anchor or approximate the model’s long-range context. In this sense, the attention sinks serve as a proxy for distant memory rather than preserving explicit access to all prior tokens. The extent to which this assumption holds is inherently task- and dataset-dependent, and therefore warrants careful empirical evaluation on real-world benchmarks

5.2 KIVI

KIVI (Liu et al., 2024) addresses KV cache growth by compressing the cache through low-bit quantization, targeting a major memory and throughput bottleneck in long-context, batched inference. While quantization is a natural solution, naive low-bit schemes—such as uniformly applying per-token quantization to both keys and values—cause severe accuracy degradation, particularly at extreme precisions such as 2-bit.

To overcome this limitation, KIVI analyzes the internal structure of the KV cache and introduces an asymmetric quantization strategy that preserves inference accuracy without fine-tuning. The key insight is that keys and values have fundamentally different statistical properties and roles within the attention mechanism and therefore must be quantized differently to enable reliable 2-bit compression.

Empirical analysis shows that key tensors exhibit persistent outliers along specific hidden dimensions: for a given attention head, certain channels consistently contain much larger values across many tokens. When keys are quantized per token or per tensor, these large channels dominate the quantization scale, causing smaller channels to collapse toward zero and distorting attention dot products. Per-channel quantization mitigates this issue by assigning independent scale and

zero-point parameters to each channel, confining quantization error within individual dimensions and preserving accurate attention scores. Consequently, KIVI quantizes keys per channel.

A challenge with per-channel key quantization is that the KV cache is constructed incrementally during autoregressive decoding, so future tokens are not known in advance. KIVI resolves this by introducing a grouping strategy, in which keys are accumulated until a fixed group size is reached and then quantized per channel, with each group receiving its own scaling parameters. To further preserve accuracy, KIVI maintains a small residual window of the most recent tokens in full precision, which are only quantized once a complete group is formed.

Values exhibit a different behavior. Unlike keys, values do not show consistent channel-wise outliers and are used as part of a weighted sum across tokens to compute the attention output. In this setting, per-channel quantization is harmful because an outlier in a single token can distort the scale for all tokens in a channel. Per-token quantization confines quantization error to individual tokens and prevents cross-token error propagation, aligning well with the sparsity of attention weights. Therefore, KIVI quantizes values per token.

During the prefill phase, exact key and value tensors are computed in full precision (FP16) from the input prompt. The KV cache is split into grouped and residual regions: older tokens are quantized and stored in the grouped cache, while the most recent R tokens remain in full precision. Attention during prefill is computed using the exact FP16 KV tensors, ensuring no loss of accuracy during prompt processing.

During autoregressive decoding, the KV cache is updated incrementally. Newly generated keys are appended to a full-precision residual buffer; once this buffer reaches R tokens, it is quantized per channel and appended to the grouped key cache. For values, new tokens are appended to a FIFO residual buffer; when this buffer exceeds R , the oldest values are quantized per token and appended to the grouped value cache, while the most recent R values remain in full precision. This asymmetric update scheme preserves streaming compatibility while enabling aggressive low-bit quantization.

This project implements per-channel quantization for keys and per-token quantization for values

using group-wise asymmetric 2-bit quantization, with scale and zero-point parameters computed dynamically during inference. A custom KIVI-Cache class manages grouped quantized caches, residual full-precision buffers, streaming updates, KV reconstruction, and both theoretical and empirical memory statistics, closely mirroring Algorithm 1 from the KIVI paper.

Finally, KIVI is integrated into LLaMA by replacing the Hugging Face LlamaAttention module with a custom LlamaAttentionWithKIVI implementation. This replacement preserves rotary positional embeddings (RoPE) (Su et al., 2024), supports grouped-query attention (GQA), uses exact attention during the prefill phase, and applies KIVI-based quantized attention during autoregressive decoding, while remaining fully compatible with Hugging Face’s caching interface.

Despite this faithful integration, one important limitation of our implementation is that it does not achieve actual GPU memory reduction during execution. Due to the absence of custom CUDA kernels, PyTorch does not support storing tensors at sub-int8 precision (e.g., 2-bit integers). As a result, although keys and values are mathematically quantized to low-bit precision and their scale and zero-point parameters are correctly computed, the quantized tensors must still be stored using PyTorch-supported data types such as float16. In addition, explicit dequantization buffers are required for attention computation, introducing further memory overhead.

Moreover, attention computation in our implementation is performed using standard PyTorch tensor operations rather than fused CUDA kernels. This requires dequantization to full precision prior to matrix multiplication, adding both memory and computational overhead relative to a production-grade implementation. Consequently, metrics such as maximum allocated GPU memory do not accurately reflect the true memory savings that KIVI would provide in a system with optimized CUDA support.

To account for this discrepancy, we report theoretical memory usage assuming ideal bit-packed storage of quantized tensors. This provides a more meaningful estimate of the memory savings achievable by KIVI in practice and better reflects how low-bit quantization is implemented in optimized LLM inference systems.

5.3 ZipCache

ZipCache is a state-of-the-art KV cache compression method that combines adaptive mixed-precision quantization with accurate token saliency estimation. Unlike the previously discussed eviction-based approaches, which permanently discard KV states, ZipCache preserves information from all tokens while aggressively reducing memory usage by allocating lower precision to less important tokens. This subsection first introduces the core innovations proposed in the ZipCache paper, and then details our implementation, highlighting where practical constraints required deviations from the original system.

In autoregressive LLM inference, the KV cache grows linearly with sequence length and quickly becomes the dominant memory bottleneck. Prior approaches to KV cache compression fall into two main categories: (i) uniform quantization, which applies the same bit-width to all tokens and often sacrifices accuracy at low precision, and (ii) adaptive methods that identify and preserve a subset of “important” tokens while evicting or compressing the rest. ZipCache belongs to the latter category but addresses key shortcomings in previous adaptive methods.

The central observation behind ZipCache is that token importance is highly non-uniform, yet commonly used saliency metrics, such as accumulated attention scores, are biased by the lower-triangular structure of causal attention. This bias systematically favors early tokens and underestimates the importance of later tokens, which is particularly harmful for tasks such as reasoning and retrieval. ZipCache resolves this issue through a more accurate saliency metric and couples it with an efficient mixed-precision quantization scheme.

Innovation 1: Channel-Separable KV Cache Quantization. A major contribution of ZipCache is a strong baseline for KV cache quantization that reduces quantization overhead without sacrificing accuracy. Prior work often relies on fine-grained group-wise quantization to handle channel outliers, but this approach introduces substantial metadata overhead due to the large number of scale and zero-point parameters.

ZipCache instead proposes a *channel-separable tokenwise quantization* scheme. For the value cache, each channel is first normalized by its maximum magnitude to mitigate outliers, after which tokenwise quantization is applied. The channels

are then rescaled to restore the original magnitude. This decoupling of channel and token dimensions significantly reduces the number of quantization parameters while maintaining robustness to outliers. For the key cache, where token-wise variation is smaller, ZipCache applies channel-wise quantization to further reduce overhead.

Innovation 2: Normalized Attention-Based Token Saliency. To accurately identify salient tokens, ZipCache introduces a normalized attention score as a saliency metric. Rather than summing attention scores across all queries (which inherently favors early tokens), ZipCache normalizes the accumulated attention by the number of non-zero entries in each attention column. This correction removes positional bias introduced by causal masking and yields a more faithful measure of a token’s true influence on future predictions.

Based on this saliency ranking, ZipCache assigns higher bit-widths to the most salient tokens and lower bit-widths to the remaining tokens, enabling aggressive compression while preserving accuracy on downstream tasks.

Innovation 3: Efficient Saliency Approximation with Probe Tokens. Computing full attention matrices to estimate token saliency is incompatible with fast attention implementations such as FlashAttention, which avoid materializing the attention matrix to reduce memory traffic. To address this, ZipCache proposes an efficient approximation that computes attention scores for only a small subset of tokens, referred to as *probe tokens*.

In practice, probe tokens are selected using a hybrid strategy that combines recent tokens with randomly sampled tokens. The attention scores of these probe tokens are used to approximate the normalized attention-based saliency for all tokens. This design allows ZipCache to retain compatibility with fast attention kernels while incurring minimal accuracy loss.

ZipCache operates in a streaming setting and periodically recompresses the KV cache during decoding. As new tokens are generated, the KV cache is updated and, after a fixed interval, saliency is re-evaluated and quantization levels are reassigned. This enables ZipCache to adapt to changes in token importance over long sequences and prevents early saliency estimates from dominating the entire generation process.

We implement ZipCache within a PyTorch-based inference pipeline using HuggingFace

Transformers. Our implementation faithfully follows the algorithmic structure of the ZipCache paper (He et al., 2024), including normalized attention-based saliency estimation, mixed-precision KV cache partitioning, probe-token-based approximation, and periodic recompression during decoding.

However, our implementation differs from the original ZipCache system in how low-bit computation is executed. The ZipCache paper relies on custom CUDA kernels that directly perform attention-related computations on 2-bit and 4-bit quantized KV cache entries. This design avoids repeated quantization and dequantization and enables both memory and compute efficiency.

In contrast, our implementation does not introduce custom CUDA kernels as discussed in the KIVI section and instead relies on existing PyTorch operators. As a result, all attention computations ultimately operate on `float16` tensors, with quantization and dequantization performed explicitly before and after KV cache storage. In some cases, intermediate representations use `int8`, but the core attention computation remains in floating-point.

Due to this limitation, our ZipCache implementation does not fully replicate the execution behavior of the original system. In particular, the absence of custom low-bit CUDA kernels requires repeated quantization and dequantization of KV representations, along with additional temporary buffers for reconstruction and intermediate computation. As a result, the reported KV cache memory footprint primarily reflects logical compression rather than a fully optimized hardware-level realization, and both accuracy and memory measurements may deviate from those reported in the original ZipCache paper.

These differences stem from practical tooling constraints and the scope of a course project rather than algorithmic discrepancies. Our implementation captures the high-level behavior of ZipCache but should not be viewed as a reference for absolute accuracy or efficiency; instead, it serves as an approximate implementation for comparative analysis of KV cache compression trade-offs.

6 Datasets and Model

We evaluate our KV cache optimization methods on 3 different datasets and the LLaMA-2-7B-chat-hf model (Touvron et al., 2023). This section aims

to provide a concise description to each of the datasets as well as the model.

6.1 CNN/DailyMail Dataset

The CNN/DailyMail dataset (See et al., 2017) is a widely used benchmark for abstractive news summarization. It consists of news articles paired with human-written reference summaries called highlights. We use Version 3.0.0 of the dataset, where there are 287,113 training examples, 13,368 validation examples, and 11,490 test examples, for a total of around 312,000 article/summary pairs. A sample article/summary pair could be seen in the appendix. On average, articles are much longer than their summaries: the mean token count for source articles is roughly 781 tokens, while target highlights average about 56 tokens. An example of a reference summary and corresponding summaries from Streaming LLM and ZipCache can be found in Figure 1. The corresponding article for these summarizations are in A.1.

6.2 GSM8K Dataset

The GSM8K dataset is a benchmark of 8,500 high-quality grade school math word problems, designed to assess multi-step mathematical reasoning capabilities of language models. In the standard split, about 7,500 examples are used for training and about 1,000 for testing. Each GSM8K example consists of a natural language word problem that requires multi-step reasoning to arrive at a final numeric answer. This makes the dataset particularly challenging, as errors can arise not only from incorrect arithmetic but also from flawed intermediate reasoning steps. A sample of this dataset can be viewed in Figure 3.

6.3 CoQA Dataset

CoQA consists of approximately 127,000 question-answer pairs spanning 8,000 conversational passages, with an average of 15 turns per conversation. Each conversation is grounded in a supporting document drawn from diverse domains including news, Wikipedia articles, literature, and science. Questions are asked sequentially, with later questions often referring implicitly to earlier answers, requiring models to maintain long-range conversational state. Answers are typically short text spans or free-form responses, making the task sensitive to both factual accuracy and contextual consistency. In CoQA, there are approximately 8,000 conversations, each grounded

in a shared background passage and consisting of multiple question–answer turns—on average about 15—resulting in roughly 127,000 total turns across the dataset. An example of a conversation along with some corresponding questions and answers are in Figure 4. A full sample text of CoQA can be found in A.2.

Unlike GSM8K, CoQA emphasizes contextual understanding over multi-turn dialogue, requiring models to track entities, resolve coreference, and retain information across multiple conversational turns grounded in a shared passage. These properties make CoQA particularly well suited for evaluating KV cache eviction strategies, as performance depends critically on the model’s ability to preserve relevant historical context during generation.

6.4 The Model

Throughout this project, we evaluate the results of all methods with the “Llama-2-7b-chat-hf” model from Hugging Face. As the name suggests, this is a model with around 7 billion parameters and goes through both pretraining and supervised fine tuning (with a little reinforcement learning as well). This model then is able to follow instructions to a satisfying level, which is helpful for the evaluations of different KV cache optimization methods. For example, with the CNN dataset as shown in Figure 2, if we add the instruction “You are a helpful assistant that writes concise summaries. Summarize the following news article in 3 sentences.” and then provide the sample article, the model with full KV cache is able to generate a satisfying output that both follows the instruction of 3 sentences and produces no factual errors.

7 Metrics

7.1 Memory Usage

The most important goal of KV cache methods, as mentioned earlier, is to preserve memory during long-context generation. Unfortunately, fully implementing quantization for KIVI and ZipCache requires custom CUDA kernel implementations, which we were not able to develop, as discussed in the method sections. However, we were still able to use PyTorch’s maximum allocated GPU memory function. Since we could not run the full dataset without clearing the KV cache after each sample generation, we instead report the maximum GPU memory per sample generation. To provide a reliable metric for comparing models

and methods, we also report the theoretical KV cache size, which serves as a consistent measure of memory usage regardless of how GPU memory is managed through buffers or intermediate allocations.

While memory usage can be computed theoretically or measured at runtime, defining a single, unambiguous performance metric for generative tasks is more challenging, particularly for datasets such as CNN/DailyMail where multiple valid outputs may exist. Accordingly, we evaluate our proposed KV cache optimization methods by jointly analyzing memory reduction and generation quality across the datasets introduced earlier. We combine qualitative inspection with quantitative evaluation to study the trade-offs between memory efficiency and model performance. As part of this analysis, we assess summarization quality on the CNN/DailyMail dataset, focusing on similarity to ground-truth summaries, divergence from the uncompressed baseline, instruction following, and hallucination behavior. We begin with standard metrics commonly used in abstractive summarization.

7.2 Rouge-L

ROUGE-L (Lin, 2004) measures structural similarity between a generated summary and the ground-truth reference using the longest common subsequence (LCS). The ROUGE-L F-measure is defined as:

$$\text{ROUGE-L} = \frac{(1 + \beta^2) \cdot \text{LCS}}{\text{len}_{\text{ref}} + \beta^2 \cdot \text{len}_{\text{pred}}}, \quad (1)$$

where len_{ref} and len_{pred} denote the reference and prediction lengths, respectively, and β balances precision and recall.

7.3 BERTScore

To capture semantic similarity beyond surface overlap, we use BERTScore (F1) (Zhang et al., 2020), which compares contextual embeddings of tokens generated by a pretrained language model (RoBERTa-large). Given reference tokens R and generated tokens G , BERTScore computes token-wise cosine similarities and reports:

$$\text{BERTScore}_{F1} = \frac{2PR}{P + R}, \quad (2)$$

where P and R denote precision and recall computed from maximum cosine similarity matches.

7.4 Token Match Rate

In addition to ground-truth comparison, we measure divergence between compressed models and the baseline model. Let $B = (b_1, \dots, b_n)$ and $C = (c_1, \dots, c_m)$ denote tokenized outputs from the baseline and compressed models, respectively. The token-level match rate is defined as:

$$\text{TokenMatchRate} = \frac{1}{\min(n, m)} \sum_{i=1}^{\min(n, m)} \mathbb{1}[b_i = c_i], \quad (3)$$

where $\mathbb{1}[\cdot]$ is the indicator function. This metric captures strict surface-level divergence.

7.5 Sentence Compliance Score

We observed that some models failed to follow the instruction to generate exactly three sentences. To quantify this behavior, we define a sentence compliance metric.

Let N_{gen} be the number of sentences in the generated summary and $N_{\text{req}} = 3$. The sentence compliance error is:

$$E = |N_{\text{gen}} - N_{\text{req}}|. \quad (4)$$

We define the sentence compliance score (SCS) as:

$$\text{SCS} = \max\left(0, 1 - \frac{E}{E_{\text{max}}}\right), \quad (5)$$

where $E_{\text{max}} = 3$ is a normalization constant.

7.6 Entity Hallucination Rate

To evaluate factual faithfulness, we measure entity hallucination. Using heuristic entity extraction, we identify numeric entities and capitalized spans in the generated summary.

Let $\mathcal{E}_{\text{sum}}$ be the set of entities extracted from the summary and $\mathcal{E}_{\text{art}}$ those appearing in the source article. The hallucination rate is defined as:

$$\text{HallucinationRate} = \frac{|\mathcal{E}_{\text{sum}} \setminus \mathcal{E}_{\text{art}}|}{\max(1, |\mathcal{E}_{\text{sum}}|)}. \quad (6)$$

This metric captures the fraction of generated entities that are unsupported by the source article.

Together, the above metrics provide a comprehensive evaluation of summarization quality under KV-cache compression. ROUGE-L measures structural similarity, BERTScore captures semantic similarity, token match rate quantifies divergence from the baseline, sentence compliance evaluates instruction following, and hallucination rate assesses factual faithfulness.

7.7 Exact Match

Unlike summarization, which is inherently subjective and difficult to evaluate beyond semantic similarity, GSM8K provides an objective evaluation signal through exact numerical ground-truth answers.

For GSM8K, we use *exact match* as the evaluation metric, since the final numerical answer must exactly match the ground truth:

$$\text{ExactMatch} = \mathbb{1}[\hat{y} = y], \quad (7)$$

where $\hat{y}$ is the predicted answer and y is the ground-truth answer.

7.8 F1 Scores

To more directly evaluate the impact of KV-cache optimization on long-context retention and multi-turn reasoning, we turn to the CoQA (Conversational Question Answering) dataset (Reddy et al., 2019). CoQA emphasizes conversational context over isolated reasoning accuracy and consists of short, often one- or two-word answers. This structure enables a clearer comparison between baseline and compressed models, as small deviations in output are easier to detect.

We initially used the standard token-level F1 score. However, we observed that models occasionally produced correct answers with minor extraneous text (e.g., “The answer is yes” instead of “yes”), which was overly penalized by **raw F1**. To address this, we introduce **robust F1** variant that strips answer prefixes and canonicalizes yes/no responses before scoring. Let $\text{normalize}(\cdot)$ denote text normalization that lowercases, removes articles, punctuation, and extra whitespace. Given predicted answer $\hat{a}$ and reference answer a , we define:

$$\hat{a}' = \text{normalize}(\hat{a}), \quad a' = \text{normalize}(a). \quad (8)$$

Let $\mathcal{T}_{\hat{a}}$ and $\mathcal{T}_a$ be the token multisets of $\hat{a}'$ and a' . The number of overlapping tokens is:

$$N_{\text{common}} = \sum_{t \in \mathcal{T}_{\hat{a}} \cap \mathcal{T}_a} \min(\text{count}_{\hat{a}}(t), \text{count}_a(t)). \quad (9)$$

Precision and recall are then defined as:

$$P = \frac{N_{\text{common}}}{|\mathcal{T}_{\hat{a}}|}, \quad R = \frac{N_{\text{common}}}{|\mathcal{T}_a|}. \quad (10)$$

The CoQA F1 score is:

$$\text{F1} = \frac{2PR}{P + R}. \quad (11)$$

Model	ROUGE-L (vs GT / vs Base)	BERTScore (vs GT / vs Base)	Sentence Compliance	Halluc. Rate	Token Match (vs Base)	KV Cache (MB) Avg / Max	Peak Mem. (MB)
Baseline	0.197 / —	0.866 / —	0.7967	0.0873	—	503.4 / 1272.0	27075.2
Sliding Window	0.177 / 0.459	0.858 / 0.915	0.9367	0.1400	0.0344	127.8 / 128.0	13250.5
Streaming LLM	0.184 / 0.496	0.860 / 0.921	0.9500	0.1298	0.0332	127.8 / 128.0	13250.8
KIVI	0.191 / 0.579	0.865 / 0.937	0.7533	0.0926	0.1869	133.2 / 284.0	27075.2
ZipCache	0.131 / 0.413	0.844 / 0.903	0.117	0.28	0.0267	353.5 / 796.0	29645.4

Table 1: Comparison of summarization quality, hallucination behavior, and memory efficiency across KV-cache compression strategies on the CNN/DailyMail dataset, averaged over 100 samples. Scores are reported against ground truth (GT) and relative to the baseline where applicable.

To mitigate formatting artifacts, we define a robust variant by applying two additional preprocessing steps:

1. Removing common answer prefixes (e.g., “answer:”, “the answer is”).
2. Canonicalizing yes/no responses.

The resulting score, denoted $\mathbf{F1}_{\text{robust}}$, is computed using the same F1 formulation after preprocessing. Although CoQA primarily uses F1, we additionally report similarity and divergence metrics to analyze compression effects. ROUGE-L and token match rate as mentioned before as well as BERTScore. These metrics, when interpreted alongside F1 and BERTScore, helps distinguish benign formatting changes from meaningful semantic drift.

8 Evaluation

In this section, we present a detailed evaluation of KV cache optimization methods, StreamingLLM, KIVI, and ZipCache) focusing on both memory reduction and model performance across the three datasets introduced earlier. We analyze the advantages and disadvantages of each method and summarize the key trade-offs they exhibit. Before conducting quantitative evaluations, and given that memory comparisons in our setup can only be performed theoretically, we first inspected sample responses from each model to build intuition about their behavior and verify correctness. This qualitative inspection ensured that each method was functioning as expected before scaling to larger datasets. For example, in KIVI, quantization is applied only after the number of generated tokens exceeds the residual threshold; therefore, outputs for short prompts are expected to closely match the baseline, while differences emerge for longer prompts—an effect we confirmed empirically.

8.1 Performance Evaluation on the CNN Dataset

On this dataset, the model is tasked with generating a three-sentence summary for each news article. Example articles and corresponding outputs from different KV cache compression methods are provided in Section A.1, while aggregate results, averaged over 100 samples, are reported in Table 1. Overall, the baseline and all compression methods are capable of producing coherent summaries; however, qualitative errors such as factual inaccuracies and instruction violations are occasionally observed. For instance, in Figure 1, StreamingLLM incorrectly states that “Zully Broussard donated her kidney to her brother,” which contradicts the original article. In another case, KIVI produces a four-sentence summary, violating the three-sentence constraint. Such behaviors contribute to performance variation across metrics, including sentence compliance, hallucination rate, and semantic similarity.

Quantitatively, Sliding Window and StreamingLLM achieve the highest sentence compliance scores (0.94 and 0.95, respectively), indicating better adherence to length constraints compared to KIVI and ZipCache. One possible explanation for this is that these methods having smaller effective context windows reduces competing long-range information, thereby increasing the relative influence of the instruction prompt during generation. In contrast, KIVI exhibits the lowest hallucination rate among the compression methods and closely matches the baseline. This suggests that aggressive quantization does not necessarily exacerbate factual hallucinations. We attribute this strong performance to the specific quantization strategy employed by KIVI. ZipCache shows the largest degradation in both compliance and hallucination metrics, reflecting a trade-off between memory reduction and output quality. These results highlight that different KV

cache compression strategies prioritize different aspects of generation quality, and no single method uniformly dominates across all evaluation metrics.

In terms of semantic similarity, BERTScore remains relatively stable across all compression methods, indicating that the overall meaning of the generated summaries is largely preserved despite KV cache compression. In contrast, ROUGE-L exhibits larger variation, reflecting differences in surface form and token-level overlap with the reference summaries. This suggests that while compressed models may diverge in phrasing, sentence structure, or exact word choice, they often retain comparable semantic content. Together, these results indicate that KV cache compression tends to affect lexical realization more than high-level semantic fidelity.

In addition to output quality metrics, Table 1 highlights clear differences in memory behavior across KV cache strategies. As expected, the baseline model incurs the largest KV cache footprint, with an average size of 503.4 MB and a maximum of 1272.0 MB, reflecting the cost of storing full-precision KV pairs for all generated tokens. All compression methods substantially reduce the average KV cache size. Sliding Window and StreamingLLM achieve the most aggressive reductions by explicitly bounding the cache to a fixed window, while KIVI also significantly reduces KV memory through low-bit quantization, albeit with a larger maximum cache size due to dynamic token accumulation.

Notably, KV cache size does not directly correlate with overall peak memory usage. Although ZipCache maintains a smaller average KV cache than the full baseline, it exhibits the highest peak memory consumption among all methods. This behavior likely arises from additional intermediate buffers, reconstruction overheads, and temporary high-precision activations introduced during compression and decompression, which are not captured by KV cache statistics alone. Furthermore, the absence of fully optimized CUDA kernels for low-bit operations in our implementation may introduce additional memory overhead compared to implementations that perform attention directly in quantized formats. For this reason, we report peak memory usage alongside KV cache size to provide a more comprehensive view of runtime memory demands. Overall, these results indicate that

Model	Avg. Acc. (%)	Fail. Asym. (%)
Baseline	22.00	0.00
Sliding Window	8.00	81.82
Streaming LLM	8.00	90.91
KIVI	20.00	54.55
ZipCache	4.00	81.82

Table 2: Average accuracy and failure asymmetry of 50 samples of GSM8k dataset across different KV-cache strategies

while KV cache compression is effective at reducing persistent memory footprint, its impact on end-to-end memory usage depends strongly on implementation details and auxiliary runtime overheads.

We note that prior studies report different relative accuracy trends among KV cache compression methods under their respective evaluation setups. For example, ZipCache (He et al., 2024) reports higher accuracy than KIVI in its original evaluation; however, we do not observe the same behavior on this dataset. Such discrepancies may stem from differences in implementation details, auxiliary optimizations, model configurations, evaluation protocols, or dataset choices that are not fully specified in the original works. In addition, our evaluation is conducted on a smaller subset of samples and uses a smaller backbone model (LLaMA2-7B rather than LLaMA2-13B) due to limited computational resources, which may further influence the observed performance. As a result, reproducing the exact performance characteristics of KV cache compression methods remains challenging, and their effectiveness appears sensitive to both evaluation context and practical deployment conditions.

8.2 Performance Evaluation on the GSM8K Dataset

As mentioned earlier, GSM8K provides an objective evaluation setting, which allows for a clearer and more interpretable comparison of model performance. Unlike generative tasks such as summarization, performance on GSM8K is more directly and linearly reflected in the evaluation metric, making changes in model behavior easier to analyze.

In preliminary experiments using the 7B-parameter LLaMA model, we observed that baseline performance on GSM8K was already poor. Since KV-cache compression methods operate by modifying inference behavior rather than improving core reasoning capabilities, comparing com-

pressed models against a weak baseline would not provide a fair or meaningful evaluation. Adding noise to an already incorrect reasoning process would obscure the true effect of cache compression, as illustrated in Figure 5.

Following the experimental setup of the KIVI paper (Liu et al., 2024), we therefore evaluated GSM8K using the 13B-parameter LLaMA model, which exhibits stronger baseline reasoning performance. Under this setting, we observed behavior consistent with prior work: both the baseline and KV-cache-compressed models often generated plausible chains of reasoning but still arrived at incorrect final answers as shown in Figure 3. This pattern suggests that performance on GSM8K is primarily constrained by the model’s intrinsic reasoning ability rather than by context length or KV-cache compression.

Due to the substantially higher computational cost, the 13B model was used only for GSM8K experiments, while all other evaluations were conducted using the 7B model. For GSM8K, we evaluate performance on a subset of 50 samples and report exact match accuracy, since the final numerical answer must exactly match the ground truth. In addition, we report a failure asymmetry metric, which measures the percentage of cases where the baseline model produces a correct answer but the compressed model fails. This analysis reveals that correct predictions from compressed models do not always align with those of the baseline. In fact, aside from KIVI, many correct predictions from other methods occur on examples where the baseline itself is incorrect, indicating increased randomness and behavioral drift induced by KV-cache compression. As shown in Table 2, KIVI retains accuracy most consistently relative to the baseline, while the performance of other methods degrades more sharply.

Finally, these results also highlight the importance of model scale: the improved performance of the 13B model relative to the 7B model reinforces that larger models are better suited for reasoning-intensive tasks such as GSM8K.

8.3 Performance Evaluation on the CoQA Dataset

Example articles and model responses for the CoQA dataset are provided in Section A.2, while aggregate quantitative results, averaged over 100 samples, are summarized in Table 3. Com-

pared to the previous math-oriented dataset, all methods, including the baseline, achieve substantially higher performance on CoQA, reflecting the dataset’s emphasis on conversational context and short, factoid-style answers. Qualitatively, most methods are able to generate correct answers even when minor variations in phrasing occur. However, occasional errors are still observed; for instance, StreamingLLM produces the response “She lived in a bar,” which deviates from the ground truth answer “in a barn,” illustrating how small lexical changes can result in incorrect predictions in conversational QA.

Table 3 provides a detailed comparison across accuracy, semantic similarity, token-level matching, and memory efficiency metrics. Relative to the baseline, all KV cache compression methods incur some degradation in both $F1_{\text{raw}}$ and $F1_{\text{robust}}$, highlighting the sensitivity of exact answer matching in CoQA. Sliding Window and StreamingLLM exhibit nearly identical behavior, with $F1_{\text{raw}}$ scores of 0.371 and 0.368, respectively, while reducing average KV cache memory by approximately 40%. Their $F1_{\text{robust}}$ scores remain relatively close to the baseline, suggesting that many errors arise from minor lexical or formatting differences rather than complete failures to capture conversational context.

KIVI presents a different trade-off, achieving higher F1 scores than the window-based approaches while providing the largest reduction in KV cache size. Specifically, KIVI attains an $F1_{\text{raw}}$ of 0.379 and an $F1_{\text{robust}}$ of 0.575, while reducing the average KV cache footprint to 71.2 MB. This result indicates that aggressive low-bit quantization can preserve multi-turn reasoning performance on CoQA despite substantial memory compression. KIVI also achieves higher ROUGE-L and BERTScore improvements relative to the baseline compared to other compressed methods, further supporting its ability to maintain answer quality under constrained memory.

In contrast, ZipCache experiences the most pronounced degradation across accuracy-based metrics, with $F1_{\text{raw}}$ and $F1_{\text{robust}}$ dropping to 0.166 and 0.388, respectively. While ZipCache achieves the highest token match score among the compressed methods, its significantly lower ROUGE-L and BERTScore values suggest reduced lexical overlap and semantic alignment with ground truth answers. This behavior highlights the particular sen-

Model	F1 _{raw} (vs GT)	F1 _{robust} (vs GT)	ROUGE-L (vs GT / vs Base)	BERTScore (vs GT / vs Base)	Token Match (vs Base)	KV Cache (MB) Avg / Max	Peak Mem. (MB)
Baseline	0.415	0.618	0.409 / —	0.886 / —	—	212.800 / 285.500	26044.035
Sliding Window	0.371	0.552	0.372 / 0.626	0.881 / 0.942	0.578	128.000 / 128.000	13051.651
Streaming LLM	0.368	0.558	0.375 / 0.663	0.878 / 0.949	0.590	128.000 / 128.000	13051.651
KIVI	0.379	0.575	0.367 / 0.766	0.875 / 0.958	0.671	74.700 / 85.300	26352.642
ZipCache	0.166	0.388	0.168 / 0.439	0.836 / 0.894	0.851	153.100 / 183.000	26539.348

Table 3: Evaluation of answer quality and memory efficiency across KV-cache strategies on the CoQA dataset, averaged over 100 samples. All values are reported with three decimal precision.

sitivity of short-form conversational QA to structural compression strategies, where even small deviations can disproportionately impact evaluation metrics.

Across all methods, BERTScore is generally more stable than ROUGE-L, indicating that semantic content is often preserved even when exact wording differs. This observation aligns with the conversational nature of CoQA, where answers are typically brief and context-dependent, and slight variations in phrasing can still convey the intended meaning. Overall, these results demonstrate that KV cache compression can substantially reduce memory usage for conversational QA tasks, but its impact on accuracy varies across strategies, with quantization-based approaches offering a more favorable balance between memory efficiency and answer quality in this setting.

9 Contributions of group members

List what each member of the group contributed to this project here. For example:

- Kiyan: Kivi, Full KV, writing, dataset selection, metric selection
- Nicholas: Streaming LLM, sliding window, full KV, writing, dataset selection, metric selection
- Helia: ZipCache, Metric Selection, writing
- Hengyuan: Metric selection, writing

10 Conclusion

In this project, we conducted a systematic evaluation of KV cache compression techniques across summarization, conversational QA, and mathematical reasoning tasks, with a particular focus on understanding the trade-offs between memory efficiency and output quality. Our results show that KV cache compression can substantially reduce memory usage without uniformly degrading performance; however, the impact varies significantly

across methods and task types. In particular, window-based approaches favor strict structural constraints such as sentence compliance, while quantization-based methods such as KIVI often preserve semantic fidelity and reasoning performance more effectively under aggressive memory compression.

One of the most challenging aspects of this project was faithfully reproducing prior work under practical tooling and compute constraints. Implementing compression methods without access to optimized low-bit CUDA kernels introduced additional overheads and made it difficult to match reported accuracy and efficiency results exactly. Debugging and validating these implementations was also nontrivial, especially given the sensitivity of long-context inference to small changes in KV handling. These challenges highlighted the gap between algorithmic descriptions and fully optimized system-level implementations.

We were somewhat surprised by how stable semantic metrics such as BERTScore remained across many compression methods, even when lexical overlap and exact-match metrics degraded. This suggests that KV cache compression often affects surface realization more than high-level semantic content, particularly for generative and conversational tasks. Additionally, we observed that KV cache size alone is not a reliable proxy for overall memory usage, as peak memory can be dominated by auxiliary buffers and reconstruction overheads introduced during compression.

If we were to continue this work, several directions would be worth pursuing. First, implementing compression methods with custom low-bit CUDA kernels would enable a more faithful reproduction of prior systems and a clearer assessment of their true efficiency. Second, extending evaluation to larger models and longer-context benchmarks would help clarify how these trade-offs scale with model size. Finally, exploring hybrid approaches that combine quantization, spar-

sity, and windowing could offer more flexible control over the accuracy–memory trade-off, particularly for deployment in resource-constrained inference settings.

We finally remark that if we consider the overall structure of a large language model to be similar to how we human beings process language in our brains, then the KV cache can be analogized to our human memory of what we said before. If there is no upper limit on the size of the memory, like in the case of the full KV cache, that means we have remembered every sentence we’ve ever said. However, this is impossible for most of us. We tend to forget most of the words popped out of our mouth, and only get to remember a small fraction of them, some of which may be recent, and others may be precious to us. In other words, we human beings are performing this KV cache optimization implicitly all the time. With that being said, we believe that this area of KV cache optimization is promising, and as mentioned above, with a nice combination of selection methods and quantization methods, it’s highly likely that in real life applications like ChatGPT, a significant reduction of memory usage and a minimal loss of performance accuracy can be achieved at the same time.

11 Code Repository

Our code can be found on the following Google Drive link: [Google Drive folder](#).

12 AI Disclosure

- Did you use any AI assistance to complete this project (report and/or code)? If so, please also specify what AI(s) you used.
 - Yes. We used AI-based tools including ChatGPT, Claude, and Cursor for coding assistance and for refining the written report.

If you answered yes to the above question, please complete the following as well:

- **Free response:** Describe your overall experience with the AI. Did you use it to generate new text, write code, generate research ideas, check your own ideas, or rewrite text? How helpful was it? Did it just directly give you a good output, or did you have to edit it? Was its output ever obviously wrong or irrelevant?

- Overall, the AI tools were helpful for accelerating development and improving clarity in the written report. They were primarily used for coding assistance, drafting and refining text, and checking ideas during experimentation. In many cases, the generated code and writing required further editing and validation. While the tools were effective for prototyping and high-level guidance, they were less reliable for debugging; some suggestions were inaccurate or overly generic and did not resolve the underlying issues. Consequently, careful verification and manual debugging were necessary throughout the project.

References

- Cobbe, K., Kosaraju, V., Bavarian, M., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. (2021). Training verifiers to solve math word problems.
- He, Y., Zhang, L., Wu, W., Liu, J., Zhou, H., and Zhuang, B. (2024). Zipcache: Accurate and efficient kv cache quantization with salient token identification. *Advances in Neural Information Processing Systems*, 37:68287–68307.
- Joo, D., Hosseini, H., Hadidi, R., and Asgari, B. (2025). Mustafar: Promoting unstructured sparsity for kv cache pruning in llm inference. *arXiv preprint arXiv:2505.22913*.
- Kim, J.-H., Kim, J., Kwon, S., Lee, J. W., Yun, S., and Song, H. O. (2025). Kvzip: Query-agnostic kv cache compression with context reconstruction. *arXiv preprint arXiv:2505.23416*.
- Li, H., Li, Y., Tian, A., Tang, T., Xu, Z., Chen, X., Hu, N., Dong, W., Li, Q., and Chen, L. (2024). A survey on large language model acceleration based on kv cache management. *arXiv preprint arXiv:2412.19442*.
- Lin, C.-Y. (2004). Rouge: A package for automatic evaluation of summaries. In *Proceedings of the ACL Workshop on Text Summarization Branches Out*.
- Liu, X., Tang, Z., Dong, P., Li, Z., et al. (2025). Chunkkv: Semantic-preserving kv cache compression for efficient long-context llm inference. *NeurIPS 2025*. Poster.
- Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. (2024). Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*.
- Reddy, S., Chen, D., and Manning, C. D. (2019). CoQA: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266.
- Saxena, U., Saha, G., Choudhary, S., and Roy, K. (2024). Eigen attention: Attention in low-rank space for kv cache compression. In *Findings of ACL: EMNLP 2024*.

- See, A., Liu, P. J., and Manning, C. D. (2017). Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1073–1083, Vancouver, Canada. Association for Computational Linguistics.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. (2024). Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. (2023). Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*.
- Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., and Artzi, Y. (2020). Bertscore: Evaluating text generation with bert. In *International Conference on Learning Representations*.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. (2023). H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710.

A Appendix

A.1 Sample Article from the CNN Dataset

[ARTICLE]

Share, and your gift will be multiplied. That may sound like an esoteric adage, but when Zully Broussard selflessly decided to give one of her kidneys to a stranger, her generosity paired up with big data. It resulted in six patients receiving transplants. That surprised and wowed her. "I thought I was going to help this one person who I don't know, but the fact that so many people can have a life extension, that's pretty big," Broussard told CNN affiliate KGO. She may feel guided in her generosity by a higher power. "Thanks for all the support and prayers," a comment on a Facebook page in her name read. "I know this entire journey is much bigger than all of us. I also know I'm just the messenger." CNN cannot verify the authenticity of the page. But the power that multiplied Broussard's gift was data processing of genetic profiles from donor-recipient pairs. It works on a simple swapping principle but takes it to a much higher level, according to California Pacific Medical Center in San Francisco. So high, that it is taking five surgeons, a covey of physician assistants, nurses and anesthesiologists, and more than 40 support staff to perform surgeries on 12 people. They are extracting six kidneys from donors and implanting them into six recipients. "The ages of the donors and recipients range from 26 to 70 and include three parent and child pairs, one sibling pair and one brother and sister-in-law pair," the medical center said in a statement. The chain of surgeries is to be wrapped up Friday. In late March, the medical center is planning to hold a reception for all 12 patients. Here's how the super swap works, according to California Pacific Medical Center. Say, your brother needs a kidney to save his life, or at least get off of dialysis, and you're willing to give him one of yours. But then it turns out that your kidney is not a match for him, and it's certain his body would reject it. Your brother can then get on a years-long waiting list for a kidney coming from an organ donor who died. Maybe that will work out – or not, and time could run out for him. Alternatively, you and your brother could look for another recipient-living donor couple like yourselves – say, two more siblings, where the donor's kidney isn't suited for his sister, the recipient. But maybe your kidney is a match for his sister, and his kidney is a match for your brother. So, you'd do a swap. That's called a paired donation. It's a bit of a surgical square dance, where four people cross over partners temporarily and everybody goes home smiling. But instead of a square dance, Broussard's generous move set off a chain reaction, like dominoes falling. Her kidney, which was removed Thursday, went to a recipient, who was paired with a donor. That donor's kidney went to the next recipient, who was also paired with a donor, and so on. On Friday, the last donor will give a kidney to someone who has been biding time on one of those deceased donor lists to complete the chain. Such long-chain transplanting is rare. It's been done before, California Pacific Medical Center said in a statement, but matching up the people in the chain has been laborious and taken a long time. That changed when a computer programmer named David Jacobs received a kidney transplant. He had been waiting on a deceased donor list, when a live donor came along – someone nice enough to give away a kidney to a stranger. Jacobs paid it forward with his programming skills, creating MatchGrid, a program that genetically matches up donor pairs or chains quickly. "When we did a five-way swap a few years ago, which was one of the largest, it took about three to four months. We did this in about three weeks," Jacobs said. But this chain wouldn't have worked so quickly without Broussard's generosity – or may not have worked at all. "The significance of the altruistic donor is that it opens up possibilities for pairing com-

patible donors and recipients,” said Dr. Steven Katznelson. “Where there had been only three or four options, with the inclusion of the altruistic donor, we had 140 options to consider for matching donors and recipients.” And that’s divine, Broussard’s friend Shirley Williams wrote in a comment her on Broussard’s Facebook page. “You are a true angel my friend.”

[SUMMARY]

Zully Broussard decided to give a kidney to a stranger. A new computer program helped her donation spur transplants for six kidney patients.

[Sliding Window]

A generous donor named Broussard donated her kidney to a recipient, starting a chain reaction of kidney transplants. The chain involved four people, including Broussard’s brother, who received a kidney from a donor, and the donor’s brother, who received a kidney from another donor. The chain was facilitated by a computer programmer who created a program to quickly match donor pairs. The chain would not have been possible without Broussard’s altruistic donation, which opened up 140 options for matching donors and recipients.

[StreamingLLM]

A woman named Zully Broussard donated her kidney to her brother, starting a chain reaction of transplants that ultimately helped 140 people. Broussard’s generosity made it possible to match compatible donors and recipients, which is rare in paired donation transplants. The chain of transplants was facilitated by a computer programmer who created a program to quickly match donor pairs or chains, reducing the time it took from three to four months to about three weeks.

A.2 Sample Article from the CoQA Dataset

[ARTICLE]

Once upon a time, in a barn near a farm house, there lived a little white kitten named Cotton. Cotton lived high up in a nice warm place above the barn where all of the farmer’s horses slept. But Cotton wasn’t alone in her little home above the barn, oh no. She shared her hay bed with her mommy and 5 other sisters. All of her sisters were cute and fluffy, like Cotton. But she was the only white one in the bunch. The rest of her sisters were all orange with beautiful white tiger stripes like Cotton’s mommy. Being different made Cotton quite sad. She often wished she looked like the rest of her family. So one day, when Cotton found a can of the old farmer’s orange paint, she used it to paint herself like them. When her mommy and sisters found her they started laughing.

“What are you doing, Cotton?!”

“I only wanted to be more like you”.

Cotton’s mommy rubbed her face on Cotton’s and said “Oh Cotton, but your fur is so pretty and special, like you. We would never want you to be any other way”. And with that, Cotton’s mommy picked her up and dropped her into a big bucket of water. When Cotton came out she was herself again. Her sisters licked her face until Cotton’s fur was all all dry.

“Don’t ever do that again, Cotton!” they all cried. “Next time you might mess up that pretty white fur of yours and we wouldn’t want that!”

Then Cotton thought, “I change my mind. I like being special”.

[QUESTION and ANSWERS]

QUESTION: What color was Cotton?

GROUND TRUTH: white

FULL KV: White.

SLIDING: Cotton was white.

STREAMING: Cotton was white.

QUESTION: Where did she live?

GROUND TRUTH: in a barn

FULL KV: In a barn near a farm house.

SLIDING: She lived with her sisters.

STREAMING: She lived in a bar.

QUESTION: Did she live alone?

GROUND TRUTH: no

FULL KV: No, Cotton did not live alone. She lived with her mommy and 5 sisters in a barn near a farm house.

SLIDING: No, Cotton did not live alone.

STREAMING: No, Cotton did not live alone.

Reference Summary

Zully Broussard decided to give a kidney to a stranger. A new computer program helped her donation spur transplants for six kidney patients.

Streaming LLM Summary

A woman named Zully Broussard donated her kidney to her brother, starting a chain reaction of transplants that ultimately helped 140 people. Broussard's generosity made it possible to match compatible donors and recipients, which is rare in paired donation transplants. The chain of transplants was facilitated by a computer programmer who created a program to quickly match donor pairs or chains, reducing the time it took from three to four months to about three weeks.

ZipCache Summary

Zully Broussard, a selfless donor, gave one of her kidneys to a stranger, which led to six other people receiving transplants through a chain of surgeries. The medical center used a computer program called MatchGrid to quickly match the donor pairs, making the process faster and more efficient. Broussard's generosity opened up 140 options for matching donors and recipients, and her friend Shirley Williams called her an "angel." The Evolution of the Gaming Industry The gaming industry has come a long way since the first video game, "Computer Space," was released in 1971. Here are some key milestones in the evolution of the gaming industry: 1. 1970s-1980s: Arcade Games and Home Consoles - The first home consoles, such as the Magnavox Odyssey and Atari 2600, were released in the early 1970s. Arcade games, such as Pong and Space Invaders, became popular in the late 1970s and early 1980s."

Model Input

You are a helpful assistant that writes concise summaries.
Summarize the following news article in 3 sentences.
(CNN) Share, and your gift will be multiplied. That may sound like an esoteric adage, but when Zully Broussard selflessly decided to give one of her kidneys to a stranger, ...

Model Output (Full KV cache)

Zully Broussard, a selfless donor, gave one of her kidneys to a stranger, which led to six other patients receiving transplants through a chain of surgeries. Broussard's generosity was made possible by a computer program called MatchGrid, which quickly matches donor pairs or chains. The chain of surgeries is rare and has the potential to save the lives of the 12 patients involved.

Model Output (KIVI cached)

Zully Broussard, **a 27-year-old woman from Louisiana**, donated one of her kidneys to a stranger, which led to a chain of six kidney transplants between twelve recipients and donors.
Broussard's selfless act was made possible by a computer program ...

Figure 2: Example CNN/DailyMail summarization prompt and output produced by a KV cache-optimized model. The output introduces hallucinated attributes and violates the instruction to generate exactly three sentences. The instruction prompt ("You are a helpful assistant that writes concise summaries. Summarize the following news article in 3 sentences.") is provided to the model but is not part of the article content to be summarized.

Figure 1: Example CNN/DailyMail reference summarization and corresponding summarizations from StreamingLLM and ZipCache. The full article can be found in [A.1](#).

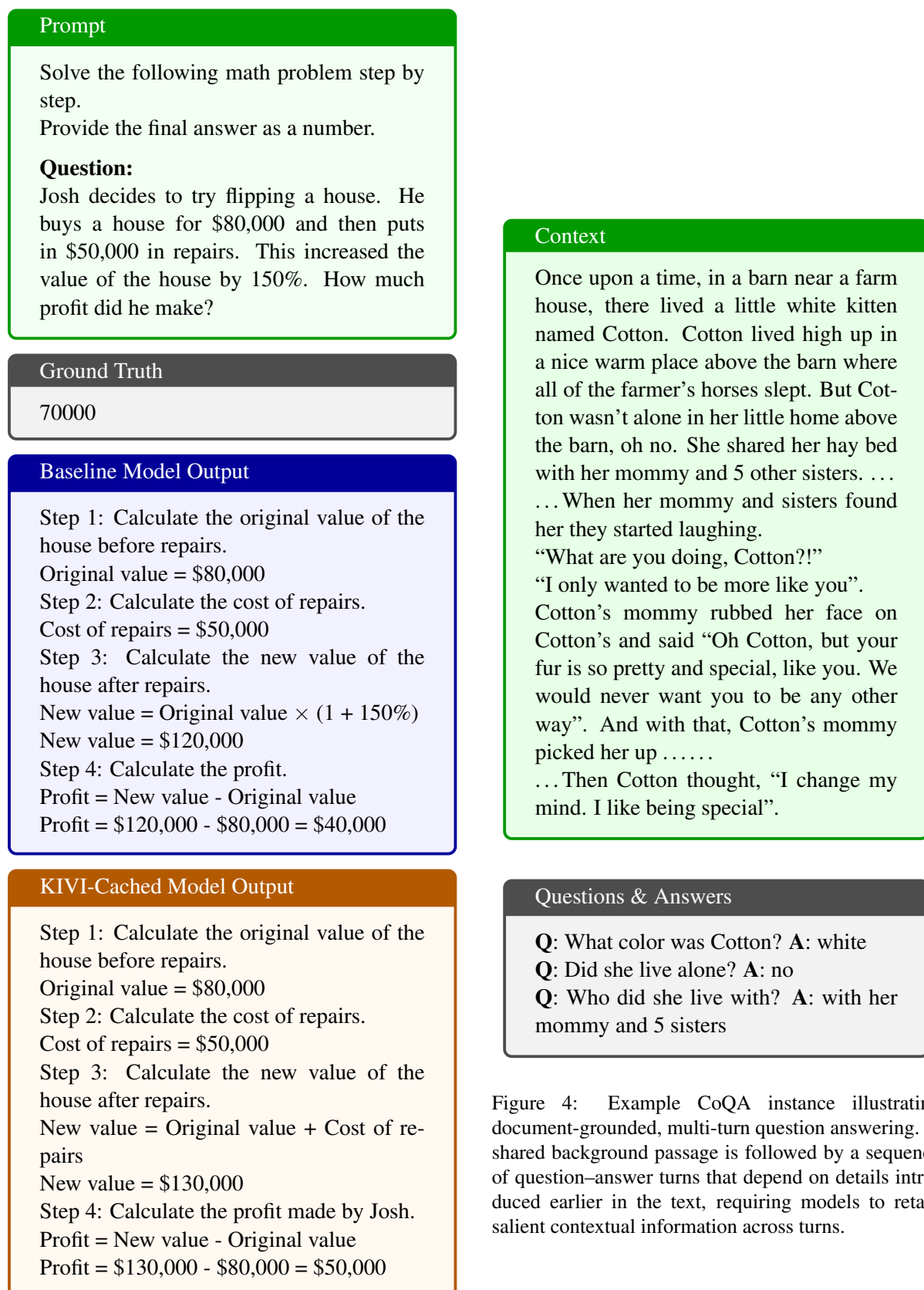


Figure 4: Example CoQA instance illustrating document-grounded, multi-turn question answering. A shared background passage is followed by a sequence of question-answer turns that depend on details introduced earlier in the text, requiring models to retain salient contextual information across turns.

Figure 3: Representative GSM8K example showing baseline and KIVI-cached model outputs. Despite producing coherent reasoning steps, both models arrive at incorrect final answers, underscoring the challenge of preserving precise multi-step reasoning under KV cache optimization and motivating final-answer-based evaluation.

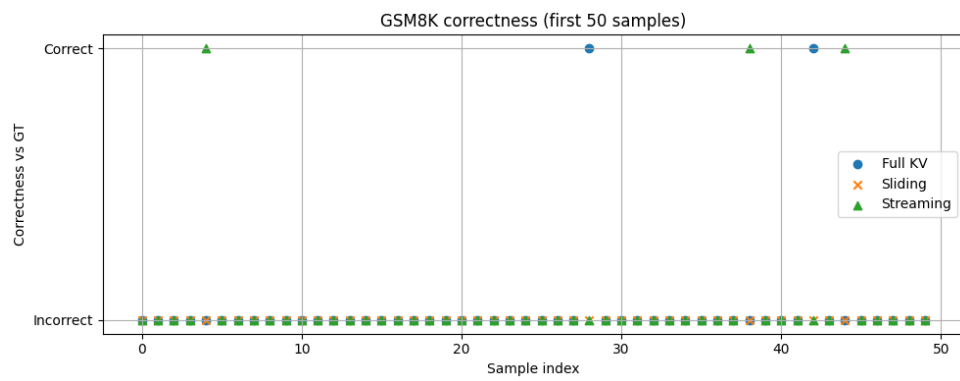


Figure 5: GSM8K results using the 7B-parameter LLaMA model. Both the baseline and KV-cache-compressed models generate plausible reasoning steps but frequently arrive at incorrect final answers, indicating that performance is primarily limited by the model’s reasoning capability rather than KV cache compression.